Manual 07 - AI Security and Lifecycle Controls

CONTROLLED PUBLICATION QA CANDIDATE

Controlled source revision: f4b19cb81daea6053de41fb3454da4170cf6f397 | Language: en | authorization and stop/rollback boundaries retained.

Security and assurance boundary: This educational manual does not prove an AI system secure, safe, compliant, certified, conformant, or free from exploitable weaknesses. Authorization, least privilege, stop/rollback, incident response, supplier risk, and evidence decisions require accountable human judgment.

Implementation and security paths

Manual 07 — AI Security and Lifecycle Implementation Paths

Essential

Use for bounded AI use cases and smaller environments. Minimum controls:

- inventory, owner, purpose, data/model/component provenance;
- threat model and approved risk treatment;
- least-privilege identities and explicit tool permissions;
- secrets protection and logging;
- secure development/change review;
- pre-deployment evaluation and security testing;
- monitoring, incident escalation, rollback/stop procedures;
- supplier/component tracking and decommissioning evidence.

Structured

Use for multiple AI systems, cloud services, RAG, external models/APIs, regulated data, or material business impact. Add:

- architecture-level trust-boundary review;
- data/model/supplier lineage;
- prompt-injection and retrieval-boundary testing;
- agent/tool authorization testing;
- adversarial evaluation/red-team scenarios;
- security telemetry and anomaly detection;
- documented release gates and exception governance;
- recurring reassessment after model/data/tool changes.

Enhanced

Use for high-impact, autonomous/agentic, safety-relevant, enterprise-scale, or highly regulated deployments. Add:

- independent technical challenge and specialized testing;
- strict privilege separation and high-risk action approval;
- sandboxing/containment and egress controls;
- continuous evaluation and attack simulation;
- supplier/component integrity verification;
- resilience, failover, stop/kill and rollback exercises;
- executive risk acceptance and material-incident governance;
- formal retirement, evidence retention, and post-incident lessons learned.

Lifecycle security route

Manual 07 memory graphic 1

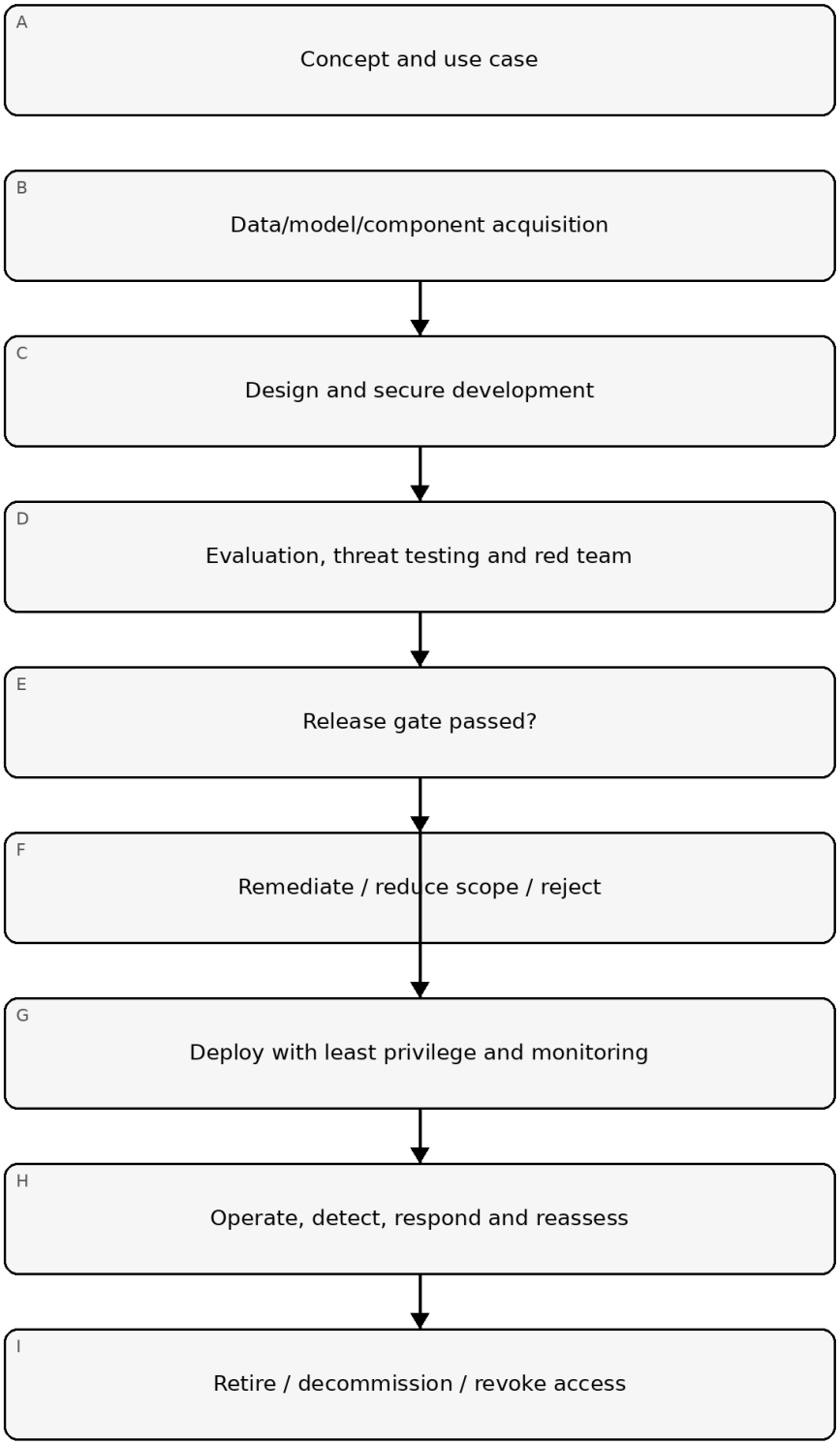

Figure 1. Implementation memory graphic

Accessible explanation: Security begins before development and continues through acquisition, design, testing, release, operation, incident response, and retirement. Failed release gates return work for remediation rather than allowing uncontrolled deployment.

Trust and authorization chain

Manual 07 memory graphic 2

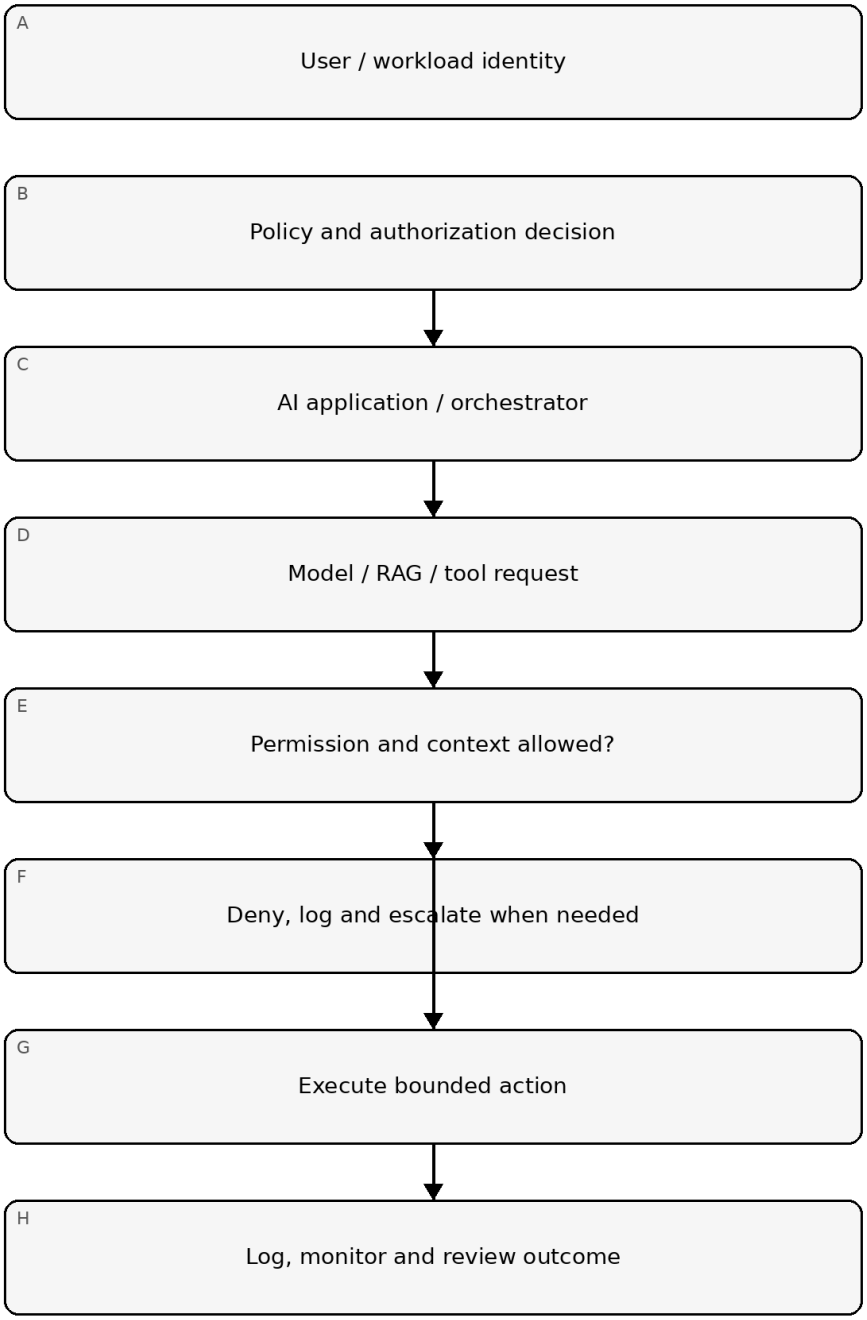

Figure 2. Implementation memory graphic

Accessible explanation: Every high-value action should pass through explicit identity, policy, authorization, and context checks. Denied actions fail closed; allowed actions remain bounded and observable.

Evidence and recovery chain

Manual 07 memory graphic 3

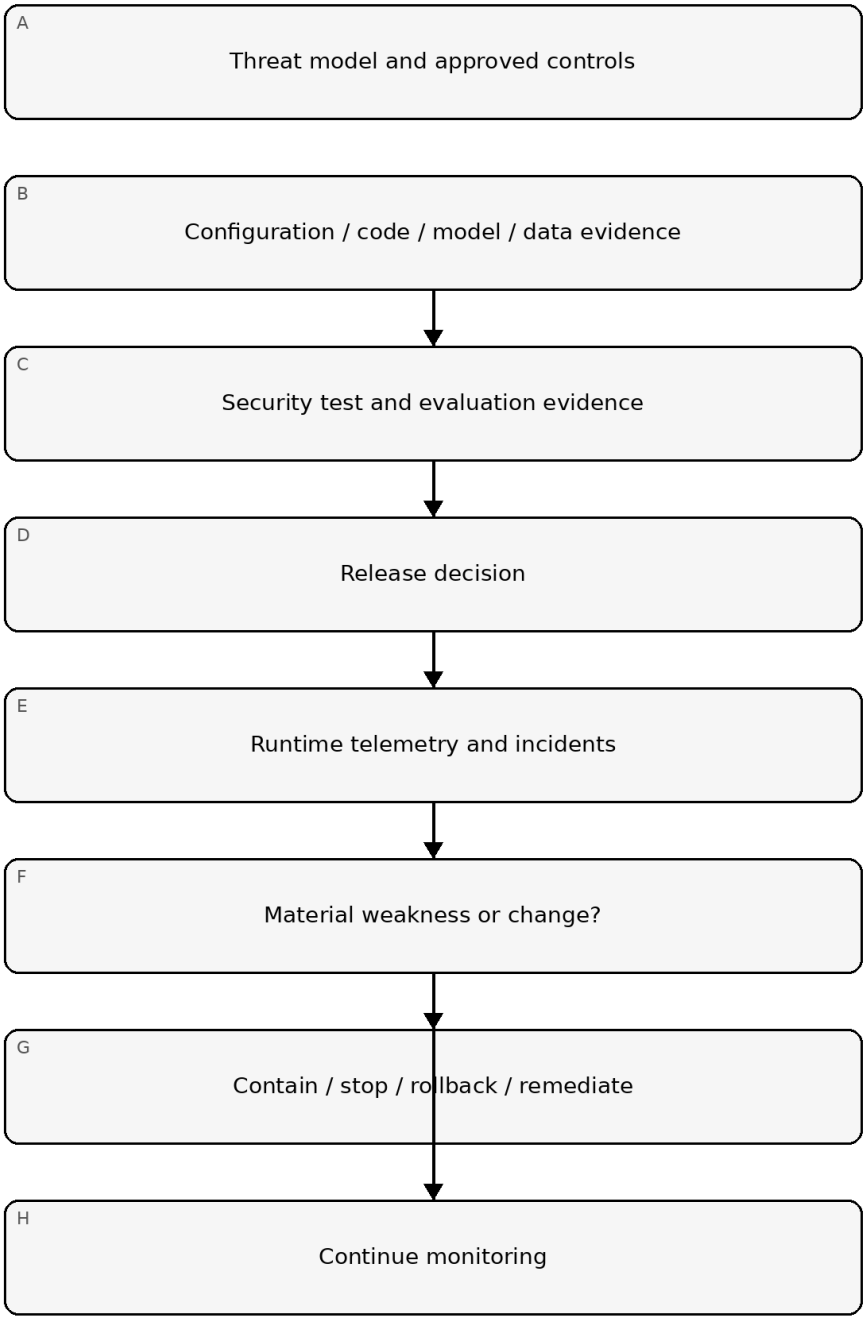

Figure 3. Implementation memory graphic

Accessible explanation: Security decisions are traceable from threat models to implementation evidence, testing, release approval, runtime telemetry, and recovery. Material weaknesses or changes trigger containment and renewed evidence rather than relying on stale approval.

Required control families

1. Governance, ownership, use-case approval, risk appetite, and change authority.
2. Asset, data, model, prompt, vector-store, tool, agent, infrastructure, and supplier inventory.
3. Threat modeling and misuse/abuse-case analysis.
4. Secure SDLC and dependency/component integrity.
5. Data provenance, integrity, privacy, classification, and retention.
6. Model/component provenance and version control.
7. Identity, authentication, authorization, least privilege, and privileged action approval.
8. Prompt injection, indirect injection, RAG poisoning, tool misuse, and agent-control testing.
9. Secrets, keys, tokens, credentials, and service-to-service trust.
10. Evaluation, security testing, red teaming, guardrails, and release criteria.
11. Monitoring, logging, detection, incident response, containment, rollback, and stop mechanisms.
12. Supplier/service risk, contract/evidence requirements, and dependency change monitoring.
13. Retirement, decommissioning, access revocation, data disposition, and evidence retention.

Security boundary

Defense in depth and testing reduce risk; they do not eliminate it. The manual must distinguish confirmed evidence, assumptions, untested areas, residual risk, and known limitations.

Controlled 32-chapter manual

Chapter 01 — Security objective and lifecycle boundary

AI security must cover the full system lifecycle rather than only the model endpoint. The controlled boundary includes use-case definition, data and model acquisition, design, development, evaluation, deployment, operation, monitoring, incident response, change, and retirement.

Each system should have a documented security objective tied to its data, actions, users, autonomy, external connectivity, and consequence of failure.

Chapter 02 — AI asset inventory

The inventory should identify models, datasets, prompts, retrieval sources, vector stores, tools, agents, APIs, service accounts, secrets, guardrails, monitoring components, hosting environments, suppliers, and critical downstream systems.

Inventory records should include owner, version, location, data classification, authentication boundary, supplier, exposure, change authority, and retirement status. Unknown components create unmanaged attack surface.

Chapter 03 — Threat modeling

Threat modeling should identify assets, trust boundaries, actors, entry points, privileges, dependencies, and plausible abuse paths. AI-specific threats should be evaluated alongside conventional application, cloud, identity, data, and supply-chain threats.

Scenarios should include malicious users, compromised retrieval content, over-privileged agents, exposed secrets, insecure APIs, unsafe tool execution, poisoned data, model or prompt disclosure, supplier compromise, and unintended autonomous behavior where relevant.

Chapter 04 — Secure development and change control

AI components should be developed and changed through controlled repositories, review, testing, dependency management, access control, and release processes. Prompt, policy, retrieval, tool, and guardrail changes can be security-significant and should not bypass change controls merely because they are not traditional code.

Material changes require reassessment of prior security evidence and may reopen release approval.

Chapter 05 — Data and model provenance

Security teams should be able to identify where models, datasets, weights, adapters, packages, prompts, and external components came from and who approved their use.

Provenance records should include origin, version, integrity evidence where available, licensing or usage boundary, supplier, approval, transformation history, and known limitations. Provenance supports trust decisions but does not prove a component is safe.

Chapter 06 — Identity, least privilege, and tool authorization

AI systems that invoke tools or external actions should use explicit identities and least-privilege permissions. The model should not receive broad credentials merely because the application needs access to multiple functions.

Authorization should be enforced outside the model whenever possible. High-impact actions should use policy checks, scoped credentials, transaction limits, human approval, or other deterministic controls appropriate to risk.

Chapter 07 — Prompt injection and untrusted content

Direct and indirect prompt injection should be treated as security threats when untrusted input can influence privileged behavior, expose sensitive information, alter system instructions, or cause unsafe tool use.

Controls can include content isolation, permission boundaries, retrieval filtering, output validation, tool allowlists, context separation, reduced privileges, confirmation steps, and monitoring. No single prompt or classifier should be treated as a complete defense.

Chapter 08 — Fail-closed release security gate

Release must fail closed when critical security evidence is missing, material findings are unresolved without approved treatment, required adversarial testing has not completed, rollback or containment is required but untested, or required human review is incomplete.

A green automated workflow supports the release decision but does not guarantee the system is secure. Final approval remains a human-controlled gate, and material post-review changes reopen affected security review.

Chapter 09 — Retrieval and knowledge-source security

Retrieval sources should be treated as externally influenced inputs. Controls should address source admission, write authority, content validation, access control, tenant separation, stale content, sensitive-data exposure, and removal.

Vector stores and indexes should inherit appropriate data classification, access, retention, logging, and backup controls.

Chapter 10 — Secrets and sensitive-data handling

Secrets should not be embedded in prompts, source code, notebooks, or model context when safer alternatives exist. Service credentials should be scoped, rotated, monitored, and stored using approved secret-management mechanisms.

Logs, traces, evaluations, and support artifacts must also be reviewed for unintended sensitive-data exposure.

Chapter 11 — Model and component supply chain

Security review should include model origin, packages, containers, adapters, datasets, APIs, plugins, safety services, and hosting dependencies. Components should be versioned and traceable so security teams can assess supplier or component change impact.

Material supplier changes should trigger reassessment rather than being silently inherited.

Chapter 12 — Evaluation and security validation

Security evaluation should use risk-based objectives and expected outcomes. Validation should cover whether access controls, data boundaries, tool permissions, retrieval controls, output handling, dependency behavior, and operational restrictions work as intended under representative and boundary conditions.

Test evidence should record configuration, scope, result, limitation, and remediation.

Chapter 13 — Independent challenge

Independent challenge should test whether assumptions and control boundaries remain valid outside normal operating conditions. The review should be authorized, bounded, and evidence-driven.

Challenge activity without remediation ownership and follow-up validation should not be represented as assurance.

Chapter 14 — Guardrails and deterministic controls

Guardrails can reduce risk but should be layered with deterministic security controls where consequences are significant. Authorization, input validation, output validation, allowlists, transaction limits, network controls, and human approval can provide stronger enforcement than model behavior alone.

Chapter 15 — Human oversight for security-sensitive actions

Human approval should be required where automated actions can create material security or business impact and the system cannot reliably constrain risk through deterministic controls.

Reviewers need enough context, time, competence, and authority to reject or stop the action. A nominal approval step without meaningful information is not effective oversight.

Chapter 16 — Pre-deployment security package

Before release, assemble the current threat model, architecture, asset inventory, validation results, open findings, supplier evidence, identity/permission review, monitoring thresholds, incident plan, rollback/stop plan, exceptions, and approvals.

The package must correspond to the exact release candidate and material changes must reopen affected evidence.

Chapter 17 — Deployment hardening

Deployment should use approved configurations, least-privilege identities, protected secrets, controlled network paths, logging, monitoring, and rollback capability appropriate to risk.

The deployed system should be compared with the validated release candidate so security-relevant drift is not introduced during promotion.

Chapter 18 — Monitoring and alerting

Monitoring should focus on indicators linked to known risk: unusual access, permission changes, repeated control failures, unexpected tool activity, sensitive-data handling, dependency changes, availability degradation, and policy exceptions.

Alerts should have owners, severity rules, escalation paths, and documented response expectations.

Chapter 19 — Logging and evidence preservation

Security-relevant logs should preserve enough context to support investigation while respecting privacy and data-minimization requirements. Useful records may include identities, timestamps, model/configuration references, tool invocations, policy decisions, retrieval references, and change events.

Retention should be defined and access to logs controlled.

Chapter 20 — Incident response

AI-related security events should integrate with the organization’s incident process. Response plans should identify containment options, evidence to preserve, supplier contacts, notification paths, recovery steps, and criteria for suspending or restricting the service.

Chapter 21 — Rollback and stop mechanisms

Systems with material operational or security impact should have tested rollback or stop mechanisms. Authority to invoke them must be explicit.

A control that exists only on paper should not be credited until it has been technically and operationally validated.

Chapter 22 — Change and configuration management

Changes to models, retrieval, prompts, system instructions, tools, permissions, data sources, hosting, guardrails, or suppliers can alter security posture. Change records should classify materiality and identify which previous validation remains valid.

Chapter 23 — Exception governance

Security exceptions should record the unmet requirement, business rationale, compensating controls, owner, residual risk, approver, expiration date, and monitoring requirement.

High-risk exceptions should not become permanent through repeated administrative extension without reassessment.

Chapter 24 — Periodic security reassessment

Periodic reassessment should examine whether threats, dependencies, access, data use, supplier state, operational behavior, and prior assumptions remain valid.

Evidence should show what was reviewed, what changed, what remains acceptable, and what additional action is required.

Chapter 25 — Supplier and dependency change

Supplier or dependency changes should be assessed for security effect before adoption. Relevant changes include hosting, model version, service architecture, data processing, subprocessors, access methods, logging, safety controls, and contractual notification commitments.

Chapter 26 — Resilience and degraded operation

Security planning should consider how the system behaves when models, APIs, retrieval, monitoring, or external services are degraded or unavailable. Fallback modes should not silently bypass security, approval, or data-protection controls.

Chapter 27 — Backup and recovery considerations

Recovery planning should identify what configurations, prompts, policies, indexes, credentials, evidence, and dependencies are needed to restore a known controlled state. Recovery procedures should be validated proportionate to criticality.

Chapter 28 — Decommissioning

Retirement should revoke identities, credentials, and integrations; disable endpoints; remove or archive data according to requirements; preserve required evidence; close supplier access; and document unresolved obligations.

Chapter 29 — Security metrics and management reporting

Metrics should be linked to decisions. Useful reporting can include material findings, exceptions, reassessment status, incident trends, dependency changes, validation coverage, overdue remediation, and control-health indicators.

Chapter 30 — Security assurance limitations

No automated test suite, control checklist, security review, or repository workflow can establish that an AI system is free from weaknesses. Assurance statements should identify the scope, time period, evidence, and limitations supporting them.

Chapter 31 — Continuous improvement

Lessons from incidents, near misses, testing, supplier changes, user feedback, and control failures should feed back into threat models, validation plans, operating controls, and training.

Chapter 32 — Manual release boundary

Before this manual is published, source verification, the complete controlled English master, technical/security review, es-419 and pt-BR semantic review, graphics/accessibility review, document and page QA, provenance, repository/security release audit, and Final Human Release Approval must be complete.

Material content changes after human approval reopen the affected gates.